Chapter 6. CPU scheduling. Full solutions

These original derivations correspond to the stable identifiers in the separate exercises PDF.

OS06-E01. Nonpreemptive scheduling

FCFS: P1[0,8), P2[8,12), P3[12,14). SJF:P3[0,2), P2[2,6), P1[6,14). Keep job identity fixed in the results.

Job	FCFS completion/wait/turnaround	SJF completion/wait/turnaround
P1	8/0/8	14/6/14
P2	12/8/12	6/2/6
P3	14/12/14	2/0/2

Mean waiting 20/3 versus 8/3 differs by 4. Mean turnaround 34/3 versus 22/3 also differs, although both finish all jobs at 14. The exchange argument assumes simultaneous arrivals, known demands, no I/O or additional switch costs. Prediction errors need not preserve the ideal ordering.

OS06-E02. Round Robin

q=2: P1[0,2), P2[2,4), P3[4,6), P1[6,8), P3[8,9), P1[9,10).

Job	Completion	Wait=completion−demand	First-CPU response
P1	10	5	0
P2	4	2	2
P3	9	6	4

Mean waiting is 13/3. Total demand is 10; there is no idle or included switch cost.

q=1: each interval has length 1.

Interval start	0	1	2	3	4	5	6	7	8	9
Job	P1	P2	P3	P1	P2	P3	P1	P3	P1	P1

Job	Completion	Waiting	First-CPU response
P1	10	5	0
P2	5	3	1
P3	8	5	2

Mean waiting remains13/3, although P2 waits 1 longer and P3 waits 1 less. Both respond earlier. Slice boundaries increase from 5 to 9; actual cross-task switches increase from 5 to 8. The final two P1 slices need no cross-task switch.

OS06-E03. Priorities and starvation

1. Construct low-priority L ready at 0 and one high-priority H_k arriving at each integer k, each demanding 1 ms. High-priority preemption runs H_k over[k, k+1), forever denying L the CPU. One observed long delay is insufficient to prove this.
2. Specify aging: priorities0–10, larger wins; L starts at 0 and gains1 every 2 ms waiting, reaching the highest level at 20 ms. Equal priorities use FIFO, new arrivals cannot jump ahead, each selected task holds the CPU for at most B ms, and switching cost is zero.
3. If q ready jobs are ahead when L is promoted and the running job has at most B left, waiting is at most20+(q+1)B ms. A common bound promised before admission also needs q≤N, yielding20+(N+1)B. Unbounded predecessors, unbounded service or unfair ties defeat the claim; the name aging is not a numeric bound.

OS06-E04 HRTICK CFS event reasoning

1. n=2≤8 gives period 6. Each slice=6×1024/2048=3 ms. Nice 0 adds one vruntime ms per real ms.

2. States follow event handling and next-current selection. Ready excludes current; min_vruntime never decreases.

t	Current	Other ready (job@vruntime)	min_vruntime
0	P1	P2@0	0
3	P2	P1@3	0
6	P1	P2@3	3
9	P3	P2@3, P1@6	3
11	P2	P1@6	3
12	P1	None	6
14	None	None	8

Equal keys follow insertion order; add no switch at 8.

3. HRTICK causes switches at 3 and 6. At 9, P3 enqueues before the old timer callback, and hrtick_update requests rescheduling; completions cause switches at 11 and 12. Thus P1[0,3), P2[3,6), P1[6,9), P3[9,11), P2[11,12), P1[12,14). Completion 14/12/11. An ordinary tick at 8 need not cause a switch.
4. P3 returns at max(0,3−3)=0. Its enqueue raises the runnable count to 3, reducing each ideal runtime from 3 ms to 2 ms; P1 has already run 3 ms in this slice. Linux 3.2 enqueue_task_fair calls hrtick_update, which sees slice−ran<0 and sets reschedule on the current task. Subsequent check_preempt_wakeup sees that rescheduling is already requested and returns; the scheduler selects P3, whose vruntime is smallest. Processing the old timer first could also request rescheduling, but this example must not attribute the decision to the wakeup comparison.

OS06-E05 Strict periodic-tick comparison

1. delta_exec=3 at 3, 4 at 4. First condition is >ideal, so 3>3 is false and 4>3 true.
2. P1[0,4), P2[4,8) completes, P1[8,9), P3[9,11), P1[11,14). At 9 min 5 places P3 at 2 and P1 key 5 leads 3>1. Completion 14/8/11.
3. Ideal runtime is the scheduling slice, not a hardware timing promise. Ordinary tick resolution and its strict condition determine observation points. In this idealized model, an enabled HRTICK requests rescheduling when its timer fires; earlier wakes and completions can alter that boundary.
4. At P3’ s enqueue, three runnable jobs reduce the ideal slice to 2 ms; P1 has already used 2.5 ms, so hrtick_update requests rescheduling. P1[6,9) first becomes [6,8.5). Then P3[8.5,10.5), P2[10.5,11.5), P1[11.5,14). Completions 14/11.5/10.5.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

OS06-E06 CPU fairness and synchronization

1. Total 1359. A slice 4.521 ms, B slice 1.479 ms; shares $1024/1359 \approx 75.35\%$,$335/1359 \approx 24.65\%$.
2. No. Two ordinary counter++ operations can interleave loads and stores even on one preemptive CPU, losing an increment in the abstract model; in C/C++ the unsynchronized conflicting non-atomic accesses are a data race with undefined behavior. CPU share does not make RMW atomic.
3. Publish ready with release store and observe it with an acquire load reading that release, or use an equivalent mutex/CV protocol. A numeric ordinary ready=1 does not establish buffer visibility. Buffer lifetime and absence of conflicting subsequent mutations are also required.
4. The same mutex around every conflicting access excludes overlap. A lock/unlock API with acquire/release semantics also establishes the corresponding happens-before edge. Publication must actually occur inside that contract, with correct waits, errors, and releases.
5. A barrier orders operations but does not admit only one thread. An atomic operation is indivisible for the specified location; its memory-order contract can publish other data. A mutex owns an entire critical section, usually with lock-acquire/unlock-release ordering. A binary semaphore initially 1 can serialize through successful wait and authorized signal but has no owner; counting semaphores bound permits. Semaphore visibility depends on the API. Fair scheduling, atomicity, exclusion, and memory ordering are four distinct guarantees.

OS06-E07 Worked solution

1. Bound and interference. Set A has $U=2/5+4/8=0.9>2(\sqrt{2}-1)\approx 0.8284$. The sufficient test is inconclusive. Iteration gives $4\to6\to8\to8$: $4+\lceil 4/5\rceil 2=6$, $4+\lceil 6/5\rceil 2=8$, $4+\lceil 8/5\rceil 2=8$. $J_{2,0}$ completes exactly at deadline 8; highest-priority $J_{1,0}$ completes at 2.

2. Initial RM trace. $[0,2)$ $J_{1,0}$; $[2,5)$ $J_{2,0}$ executes 3, leaving 1; $[5,7)$ $J_{1,1}$; $[7,8)$ $J_{2,0}$ completes; $[8,10)$ $J_{2,1}$ executes 2, leaving 2. The first two τ_1 jobs complete2/7 against 5/10. $J_{2,0}$ completes 8 against 8. This checks the prefix, not every future RM job.

3. EDF events. Parentheses give (remaining, absolute deadline). Decisions follow completions and releases at that time.

t	Ready jobs	Execution until next event
0	$J_{1,0}$(2,5), $J_{2,0}$(4,8)	$J_{1,0}$:0–2
2	$J_{2,0}$(4,8)	$J_{2,0}$:2–5
5	$J_{2,0}$(1,8), $J_{1,1}$(2,10)	$J_{2,0}$:5–6; continue
6	$J_{1,1}$(2,10)	$J_{1,1}$:6–8
8	$J_{2,1}$(4,16)	$J_{2,1}$:8–10
10	$J_{2,1}$(2,16), $J_{1,2}$(2,15)	$J_{1,2}$:10–12; preempt
12	$J_{2,1}$(2,16)	$J_{2,1}$:12–14
14	Empty	Idle:14–15
15	$J_{1,3}$(2,20)	$J_{1,3}$:15–16; one remains

Jobs due by 16: $J_{1,0}$/$J_{1,1}$/$J_{1,2}$ complete2/8/12 against 5/10/15; $J_{2,0}$/$J_{2,1}$ complete6/14 against 8/16. All pass. At 8, $J_{1,1}$ completes before new $J_{2,1}$ enters. A new $J_{2,2}$ at 16 does not alter the old $J_{2,1}$'s completed status. $U=0.9\leq 1$ and D=T allow the ideal EDF theorem to certify the infinite sequence; the finite prefix alone cannot.

4. Sufficient, not necessary. Set B has U=1. RM executes $J_{1,0}$:[0,1), $J_{2,0}$:[1,2), $J_{1,1}$:[2,3), $J_{2,0}$:[3,4). Completions 1/3/4 meet2/4/4. At 4 no job remains and synchronous releases recur, proving repetition every 4 units. Thus RM can succeed above0.8284.

5. Short deadlines and assumptions. Set C has U=0.4 but each task contributes $dbf(3)=\max(0,\lfloor(3-3)/10\rfloor+1)\cdot 2=2$, so demand 4>3. Every policy misses a deadline. A meets D=T; C does not. Added lock blocking can prevent service to the earliest-due ready job, invalidating direct reuse of the old busy-interval proof. Supply blocking bounds and resource-protocol analysis; naming priority inheritance is not a new guarantee.

OS06-E08. An arrival need not preempt

SRTF:P1[0,2), P2[2,5), P3[5,6), P1[6,11). At 2, P1 has 5 left versus P2's3. At 4, P2 and P3 both have1; keeping current causes no switch.

Job	Completion f	Turnaround f−a	Waiting f−a−b	First-CPU response
P1	11	11	4	0
P2	5	3	0	0
P3	6	2	1	1

Mean waiting is 5/3. Continuing P2 across an arrival is consistent with the preemptive policy and its tie rule.

Nonpreemptive SJF:P1[0,7), P3[7,8), P2[8,11). Only P1 is ready initially; later arrivals cannot interrupt it.

Job	Completion f	Turnaround f−a	Waiting f−a−b	First-CPU response
P1	7	7	0	0
P2	11	9	6	6
P3	8	4	3	3

Mean waiting is 3. SRTF improves this mean but delays P1; individual and aggregate outcomes differ.

OS06-E09. Interrupted MLFQ budgets

A exhausts Q0 at 2 and moves to Q1. It starts its Q1 budget of 4 only after B completes in Q0.

Interval	Job/level	Remaining demand	Remaining budget/outcome
0–2	A/Q0	5	0; demote to Q1
2–4	B/Q0	0	Complete
4–5	A/Q1	4	3; preempted by C
5–6	C/Q0	0	Complete
6–9	A/Q1	1	0; demote to Q2
9–10	A/Q2	0	Complete

Job	Completion	Waiting=f−arrival−demand	First-CPU response
A	10	3	0
B	4	1	1
C	6	0	0

Resetting A's budget to 4 at 6 lets it finish in Q1 over 6–10, without reaching Q2. Completion happens to be unchanged here, but the level and budget differ. Another Q1 job could change later selections; equal final completion does not make the policies equivalent.

References and attribution

Lin Cing, handwritten operating systems notes. Silberschatz, Galvin and Gagne, Operating System Concepts, 9th ed., Chapter 6. Linux 3.2 scheduler source; Linux 3.2 scheduler features.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study